Number of Closed Islands

Problem Statement

Given a 2D grid consists of 0s (land) and 1s (water). An island is a maximal 4-directionally connected group of 0s, and a closed island is an island totally surrounded by 1s on all left, top, right, and bottom sides.

Return the number of closed islands.

Example 1:
Input:
grid = [
[1,1,1,1,1,1,1,0],
[1,0,0,0,0,1,1,0],
[1,0,1,0,1,1,1,0],
[1,0,0,0,0,1,0,1],
[1,1,1,1,1,1,1,0]
]
Output: 2

Example 2:
Input:
grid = [
[0,0,1,0,0],
[0,1,0,1,0],
[0,1,1,1,0]
]
Output: 1

Example 3:
Input:
grid = [
[1,1,1,1,1,1,1],
[1,0,0,0,0,0,1],
[1,0,1,1,1,0,1],
[1,0,1,0,1,0,1],
[1,0,1,1,1,0,1],
[1,0,0,0,0,0,1],
[1,1,1,1,1,1,1]
]
Output: 2

Constraints:
1 <= grid.length, grid[0].length <= 100
0 <= grid[i][j] <= 1

Approach

A closed island is a group of 0s that does not touch the boundary of the grid.

First, remove every island connected to the boundary by changing its 0s to 1s. Any land remaining after this step must be part of a closed island.

Then scan the grid. Every time an unvisited 0 is found, it is the start of one closed island. Use DFS to mark the entire island as visited and increase the count by 1.

Algorithm:
1. For every boundary cell, if it contains 0, run DFS to mark its entire connected island as water.
2. Iterate through the interior cells.
3. When a 0 is found, increment the closed-island count and use DFS to mark its whole island.
4. Return the count.

Example:
For Example 1, islands touching the top, right, or bottom boundary are removed first. The remaining two islands are completely surrounded by water, so the answer is 2.

Java Solution

```
class Solution {
    private final int[] rowDirections = {-1, 1, 0, 0};
    private final int[] columnDirections = {0, 0, -1, 1};

    public int closedIsland(int[][] grid) {
        int rows = grid.length;
        int columns = grid[0].length;

        for (int row = 0; row < rows; row++) {
            removeOpenIslands(grid, row, 0);
            removeOpenIslands(grid, row, columns - 1);
        }

        for (int column = 0; column < columns; column++) {
            removeOpenIslands(grid, 0, column);
            removeOpenIslands(grid, rows - 1, column);
        }

        int closedIslands = 0;

        for (int row = 1; row < rows - 1; row++) {
            for (int column = 1; column < columns - 1; column++) {
                if (grid[row][column] == 0) {
                    closedIslands++;
                    removeOpenIslands(grid, row, column);
                }
            }
        }

        return closedIslands;
    }

    private void removeOpenIslands(int[][] grid, int row, int column) {
        if (row < 0 || row >= grid.length
                || column < 0 || column >= grid[0].length) {
```

```
            return;
        }

        if (grid[row][column] != 0) {
            return;
        }

        grid[row][column] = 1;

        for (int direction = 0; direction < 4; direction++) {
            removeOpenIslands(
                    grid,
                    row + rowDirections[direction],
                    column + columnDirections[direction]);
        }
    }
}
```

Complexity

Time Complexity: O(rows * columns), since every cell is visited at most a constant number of times.

Space Complexity: O(rows * columns) in the worst case because of the DFS recursion stack.